

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

—o0o—



BÁO CÁO BÀI TẬP LỚN

Môn học: KHO DỮ LIỆU VÀ HỆ HỖ TRỢ QUYẾT ĐỊNH
(CO4031)

GVHD: Bùi Tiến Đức
Học kỳ - Lớp: HK252 - L01, L02

Sinh viên thực hiện	Mã số sinh viên
Nguyễn Văn A	231xxxx
Trần Văn B	231yyyy
Lê Thị C	231zzzz

TP. Hồ Chí Minh, Ngày 3 tháng 5 năm 2026

Ngày 3 tháng 5 năm 2026

Mục lục

1	Giới thiệu	2
2	Tiền xử lý Dữ liệu và Kiểm tra Chất lượng	2
2.1	Kiểm tra Chất lượng Dữ liệu	2
2.2	Chuyển đổi Dữ liệu	8
3	Vùng Lưu trữ Tạm (Staging Area) và Đường ống ETL	8
3.1	Lưu trữ Cơ sở dữ liệu	8
3.2	Triển khai ETL	10
4	Thiết kế Kho Dữ Liệu: Lược đồ Hình sao (Star Schema)	15
4.1	Các Bảng Chiều (Dimension Tables)	15
4.2	Bảng Sự kiện (Fact Table)	16
5	Phân tích OLAP	19
5.1	Truy vấn 1: Tỷ lệ Lỗi theo Loại Chất lượng Sản phẩm	19
5.2	Truy vấn 2: Xu hướng Lỗi Hàng tháng	20
5.3	Truy vấn 3: Phân phối Lỗi theo Loại Lỗi	21
5.4	Truy vấn 4: Tình trạng Máy vs Tỷ lệ Lỗi	21
6	Kết luận	21

1 Giới thiệu

Tài liệu này là báo cáo chi tiết về các bước đã thực hiện để xây dựng một Kho dữ liệu (Data Warehouse) cho dữ liệu sản xuất và bảo trì dự đoán. Dự án bao gồm việc trích xuất dữ liệu hoạt động của máy móc thô, tiền xử lý dữ liệu, tải dữ liệu vào vùng lưu trữ tạm (staging area) thông qua một đường ống ETL, xây dựng Lược đồ Hình sao (Star Schema), và cuối cùng là thực thi các truy vấn Phân tích Xử lý Trực tuyến (OLAP) để rút ra thông tin tình báo kinh doanh (business intelligence).

2 Tiền xử lý Dữ liệu và Kiểm tra Chất lượng

Tập dữ liệu thô bao gồm 10,000 bản ghi về hoạt động của máy với 14 cột ban đầu, theo dõi các thuộc tính như nhiệt độ không khí, nhiệt độ quá trình, tốc độ quay, mô-men xoắn, độ mòn của dao cụ, và các trạng thái lỗi.

UDI	Product ID	Type	Air temper	Process te	Rotational	Torque[Nr	Toolwear	Machine fa	TWF	HDF	PWF	OSF	RNF
1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
5	L47184	L	298.2	308.7	1408	40	9	0	0	0	0	0	0
6	M14865	M	298.1	308.6	1425	41.9	11	0	0	0	0	0	0
7	L47186	L	298.1	308.6	1558	42.4	14	0	0	0	0	0	0
8	L47187	L	298.1	308.6	1527	40.2	16	0	0	0	0	0	0
9	M14868	M	298.3	308.7	1667	28.6	18	0	0	0	0	0	0
10	M14869	M	298.5	309	1741	28	21	0	0	0	0	0	0
11	H29424	H	298.4	308.9	1782	23.9	24	0	0	0	0	0	0
12	H29425	H	298.6	309.1	1423	44.3	29	0	0	0	0	0	0
13	M14872	M	298.6	309.1	1339	51.1	34	0	0	0	0	0	0
14	M14873	M	298.6	309.2	1742	30	37	0	0	0	0	0	0
15	L47194	L	298.6	309.2	2035	19.6	40	0	0	0	0	0	0
16	L47195	L	298.6	309.2	1542	48.4	42	0	0	0	0	0	0
17	M14876	M	298.6	309.2	1311	46.6	44	0	0	0	0	0	0
18	M14877	M	298.7	309.2	1410	45.6	47	0	0	0	0	0	0
19	H29432	H	298.8	309.2	1306	54.5	50	0	0	0	0	0	0

Hình 1: Ví dụ dữ liệu thô và các bước tiền xử lý ban đầu

2.1 Kiểm tra Chất lượng Dữ liệu

Bước kiểm tra ban đầu được thực hiện bằng Python (pandas).

- **Giá trị bị thiếu (Missing Values):** Tìm thấy 0 giá trị bị thiếu trên tất cả các cột.
- **Giá trị lặp lại (Duplicates):** Nhận diện 0 dòng lặp lại.
- **Giá trị ngoại lai (Outliers):** Sử dụng phương pháp Khoảng tứ phân vị (Interquartile Range - IQR $\times 3$). Đã phát hiện và loại bỏ 106 giá trị ngoại lai cực đoan từ cột **Rotational speed [rpm]** (khoảng: 856.0 - 2179.0).

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import os
6 import warnings
7
8 warnings.filterwarnings('ignore')
9
10 # =====
11 # PATHS
12 # =====
13
14 RAW_PATH = r"C:\co4031_project\data\raw\ai4i2020.csv"
```

```

15 CLEANED_PATH = r"C:\co4031_project\data\cleaned\machine_data_cleaned.csv"
16 DIST_PATH    = r"C:\co4031_project\data\cleaned\eda_distributions.png"
17 HEATMAP_PATH = r"C:\co4031_project\data\cleaned\correlation_heatmap.png"
18
19 os.makedirs(os.path.dirname(CLEANED_PATH), exist_ok=True)
20
21 # =====
22 # SECTION 1: Load Raw Data
23 # =====
24
25 print("=" * 60)
26 print(" STEP 1: Loading Raw Data ")
27 print("=" * 60)
28
29 raw = pd.read_csv(RAW_PATH)
30
31 print(f"Shape           : {raw.shape}")
32 print(f"Rows            : {raw.shape[0]}")
33 print(f"Columns           : {raw.shape[1]}")
34 print()
35 print("First 5 rows:")
36 print(raw.head())
37 print()
38 print("Column data types:")
39 print(raw.dtypes)
40 print()
41
42 # =====
43 # SECTION 2: Data Quality Inspection
44 # =====
45
46 print("=" * 60)
47 print(" STEP 2: Data Quality Inspection ")
48 print("=" * 60)
49
50 missing = raw.isnull().sum()
51 print("Missing values per column:")
52 print(missing)
53 print(f"Total missing      : {missing.sum()}")
54 print()
55
56 n_dup = raw.duplicated().sum()
57 print(f"Duplicate rows     : {n_dup}")
58 print()
59
60 numeric_cols = [
61     'Air temperature [K]',
62     'Process temperature [K]',
63     'Rotational speed [rpm]',
64     'Torque [Nm]',
65     'Tool wear [min]'
66 ]
67
68 print("Descriptive statistics (numeric columns):")
69 print(raw[numeric_cols].describe().round(2))
70 print()
71
72 # =====
73 # SECTION 3: Cleaning (work on a copy raw is untouched)
74 # =====
75
76 print("=" * 60)
77 print(" STEP 3: Data Cleaning ")

```

```

78 print("=" * 60)
79
80 df = raw.copy()
81
82 # 3a: Remove duplicates
83 before = len(df)
84 df = df.drop_duplicates()
85 print(f"Duplicate rows removed      : {before - len(df)}")
86
87 # 3b: Fill missing values with column median
88 for col in numeric_cols:
89     n_null = df[col].isnull().sum()
90     if n_null > 0:
91         med = df[col].median()
92         df[col] = df[col].fillna(med)
93         print(f"    Filled {n_null} nulls in '{col}' with median={med:.2f}")
94 print("Missing value handling      : complete")
95
96 # 3c: Remove physically impossible values
97 before = len(df)
98 df = df[df['Air temperature [K]'] > 0]
99 df = df[df['Process temperature [K]'] > 0]
100 df = df[df['Rotational speed [rpm]'] > 0]
101 df = df[df['Torque [Nm]'] >= 0]
102 df = df[df['Tool wear [min]'] >= 0]
103 print(f"Impossible records removed: {before - len(df)}")
104
105 # 3d: IQR outlier removal (factor=3 only extreme outliers)
106 def remove_iqr_outliers(df, col, factor=3.0):
107     Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
108     IQR = Q3 - Q1
109     lo, hi = Q1 - factor * IQR, Q3 + factor * IQR
110     before = len(df)
111     df = df[(df[col] >= lo) & (df[col] <= hi)]
112     print(f"    '{col}': removed {before - len(df)} outliers    [{lo:.1f}    {hi:.1f}]")
113     return df
114
115 print("Removing extreme outliers (IQR x3):")
116 for col in ['Rotational speed [rpm]', 'Torque [Nm]']:
117     df = remove_iqr_outliers(df, col)
118
119 print(f"Records after cleaning      : {len(df)}")
120 print()
121
122 # =====
123 # SECTION 4: Transformation & Feature Engineering
124 # =====
125
126 print("=" * 60)
127 print(" STEP 4: Data Transformation ")
128 print("=" * 60)
129
130 # 4a: Kelvin      Celsius
131 df['Air_Temp_C'] = df['Air temperature [K]'] - 273.15
132 df['Process_Temp_C'] = df['Process temperature [K]'] - 273.15
133 print("Converted: temperatures K      C")
134
135 # 4b: Temperature differential
136 df['Temp_Differential_C'] = df['Process_Temp_C'] - df['Air_Temp_C']
137 print("Created      : Temp_Differential_C")
138
139 # 4c: Mechanical power (W = Nm * rad/s)

```

```

140 df['Power_W'] = df['Torque [Nm]'] * (2 * np.pi * df['Rotational speed [rpm]'] /
141     60)
142 print("Created : Power_W")
143 # 4d: Encode quality type L=0 M=1 H=2
144 type_map = {'L': 0, 'M': 1, 'H': 2}
145 df['Type_Encoded'] = df['Type'].map(type_map)
146 print("Created : Type_Encoded (L=0, M=1, H=2)")
147
148 # 4e: Human-readable failure label
149 def get_failure_type(row):
150     if row['TWF'] == 1: return 'Tool Wear Failure'
151     elif row['HDF'] == 1: return 'Heat Dissipation Failure'
152     elif row['PWF'] == 1: return 'Power Failure'
153     elif row['OSF'] == 1: return 'Overstrain Failure'
154     elif row['RNF'] == 1: return 'Random Failure'
155     else: return 'No Failure'
156
157 df['Failure_Type'] = df.apply(get_failure_type, axis=1)
158 print("Created : Failure_Type")
159
160 # 4f: Rename ALL columns to snake_case
161 rename_map = {
162     'UDI' : 'record_id',
163     'Product ID' : 'product_id',
164     'Type' : 'quality_type',
165     'Air temperature [K]' : 'air_temp_k',
166     'Process temperature [K]' : 'process_temp_k',
167     'Rotational speed [rpm]' : 'rotational_speed_rpm',
168     'Torque [Nm]' : 'torque_nm',
169     'Tool wear [min]' : 'tool_wear_min',
170     'Machine failure' : 'machine_failure',
171     'TWF' : 'failure_twf',
172     'HDF' : 'failure_hdf',
173     'PWF' : 'failure_pwf',
174     'OSF' : 'failure_osf',
175     'RNF' : 'failure_rnf',
176     'Air_Temp_C' : 'air_temp_c',
177     'Process_Temp_C' : 'process_temp_c',
178     'Temp_Differential_C' : 'temp_differential_c',
179     'Power_W' : 'power_w',
180     'Type_Encoded' : 'type_encoded',
181     'Failure_Type' : 'failure_type',
182 }
183 df = df.rename(columns=rename_map)
184 print("Renamed : all columns snake_case")
185 print()
186
187 # Sanity check confirm the two critical columns exist before saving
188 assert 'type_encoded' in df.columns, "BUG: type_encoded missing after rename"
189 assert 'failure_type' in df.columns, "BUG: failure_type missing after rename"
190 print("Sanity check: type_encoded failure_type ")
191 print()
192
193 # =====
194 # SECTION 5: Feature Correlation Analysis
195 # =====
196
197 print("=" * 60)
198 print(" STEP 5: Feature Correlation Analysis ")
199 print("=" * 60)
200
201 feature_cols = [

```

```

202     'air_temp_k',
203     'process_temp_k',
204     'rotational_speed_rpm',
205     'torque_nm',
206     'tool_wear_min',
207     'temp_differential_c',
208     'power_w',
209 ]
210
211 print("Correlation with machine_failure:")
212 corr = df[feature_cols + ['machine_failure']].corr()['machine_failure']
213 print(corr.drop('machine_failure').sort_values(ascending=False).round(4))
214 print("\nAll 7 features retained (domain knowledge).")
215 print()
216
217 # =====
218 # SECTION 6: Save Cleaned Dataset
219 # =====
220
221 print("=" * 60)
222 print(" STEP 6: Saving Cleaned Dataset ")
223 print("=" * 60)
224
225 df.to_csv(CLEANED_PATH, index=False)
226
227 print(f"Saved to      : {CLEANED_PATH}")
228 print(f"Final shape   : {df.shape}")
229 print()
230 print("All columns in cleaned file:")
231 for c in df.columns:
232     print(f" {c}")
233 print()
234 print("Machine failure distribution:")
235 dist = df['machine_failure'].value_counts(normalize=True).round(4) * 100
236 print(dist.rename({0: 'Normal (%)', 1: 'Failure (%)'}))
237 print()
238 print("Failure type distribution:")
239 print(df['failure_type'].value_counts())
240 print()
241 print("Type encoded distribution:")
242 print(df['type_encoded'].value_counts().sort_index())
243 print()
244
245 # =====
246 # SECTION 7: Visualisations
247 # =====
248
249 print("=" * 60)
250 print(" STEP 7: Visualisations ")
251 print("=" * 60)
252
253 # --- Distribution plots ---
254 plot_features = [
255     ('air_temp_k',      'Air Temperature (K)'),
256     ('process_temp_k', 'Process Temperature (K)'),
257     ('rotational_speed_rpm', 'Rotational Speed (RPM)'),
258     ('torque_nm',       'Torque (Nm)'),
259     ('tool_wear_min',   'Tool Wear (min)'),
260     ('power_w',         'Power (W)'),
261 ]
262
263 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
264 fig.suptitle('Feature Distributions by Machine Failure Status', fontsize=14,

```

```

fontweight='bold')
265
266 for ax, (col, label) in zip(axes.flatten(), plot_features):
267     normal = df[df['machine_failure'] == 0][col]
268     failure = df[df['machine_failure'] == 1][col]
269     ax.hist(normal, bins=40, alpha=0.6, label='Normal', color='steelblue')
270     ax.hist(failure, bins=40, alpha=0.7, label='Failure', color='tomato')
271     ax.set_title(label, fontsize=10)
272     ax.set_xlabel(label)
273     ax.set_ylabel('Count')
274     ax.legend(fontsize=8)
275
276 plt.tight_layout()
277 plt.savefig(DIST_PATH, dpi=150, bbox_inches='tight')
278 plt.close()
279 print(f"Saved: {DIST_PATH}")
280
281 # --- Correlation heatmap ---
282 fig2, ax2 = plt.subplots(figsize=(10, 8))
283 corr_matrix = df[feature_cols + ['machine_failure']].corr()
284 sns.heatmap(
285     corr_matrix, annot=True, fmt='.2f',
286     cmap='coolwarm', center=0, ax=ax2, linewidths=0.5
287 )
288 ax2.set_title('Feature Correlation Heatmap', fontsize=13, fontweight='bold')
289 plt.tight_layout()
290 plt.savefig(HEATMAP_PATH, dpi=150, bbox_inches='tight')
291 plt.close()
292 print(f"Saved: {HEATMAP_PATH}")

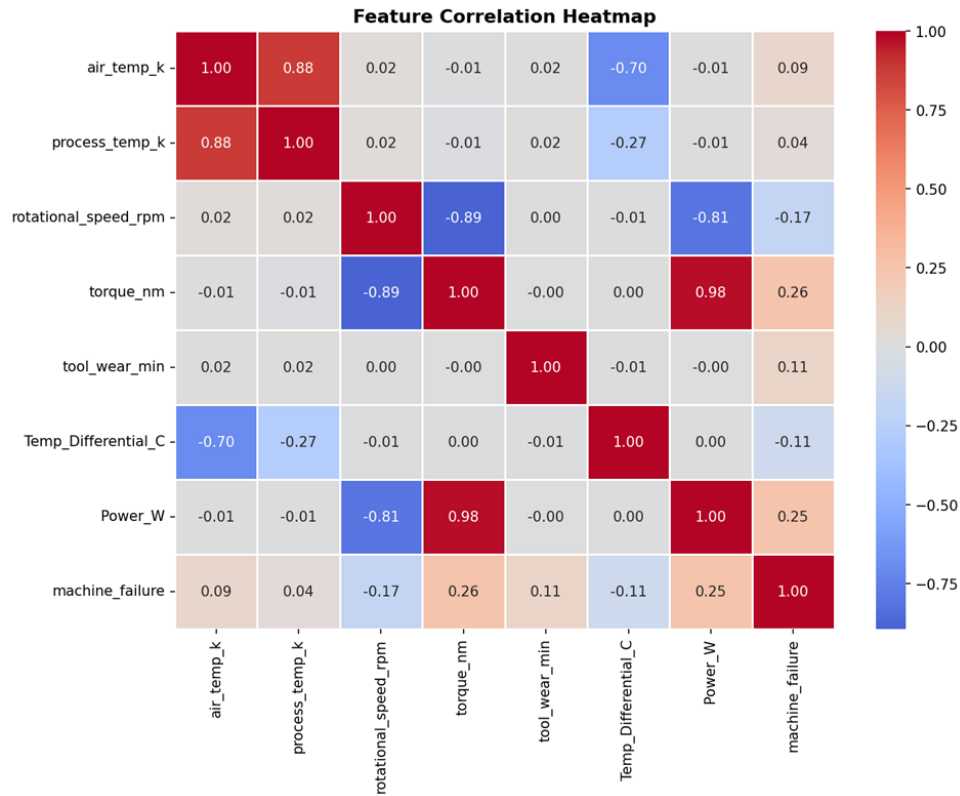
```

Listing 1: Kiểm tra toàn vẹn và loại bỏ giá trị ngoại lai bằng IQR

Tập dữ liệu đã làm sạch cuối cùng giữ lại 9,894 bản ghi.



Hình 2: Biểu đồ phân phối của các đặc trưng sau khi chuyển đổi



Hình 3: Bản đồ nhiệt (Heatmap) thể hiện độ tương quan của các đặc trưng

2.2 Chuyển đổi Dữ liệu

Một số phép chuyển đổi theo đặc thù miền (domain-specific) đã được áp dụng để làm phong phú tập dữ liệu:

1. **Chuyển đổi Nhiệt độ:** Nhiệt độ Không khí và Quá trình được chuyển đổi từ Kelvin sang độ C (trừ đi 273.15).
2. **Khai phá Đặc trưng (Feature Engineering):**
 - Temp_Differential_C: Tính bằng Nhiệt độ Quá trình - Nhiệt độ Không khí.
 - Power_W: Công suất cơ học được tính bằng Mô-men xoắn và Tốc độ quay.
3. **Mã hóa Phân loại:** Cột Type (L, M, H) được mã hóa lần lượt thành 0, 1, 2.
4. **Tạo Nhãn (Label Generation):** Một cột Failure_Type duy nhất được suy ra từ các cờ lỗi nhị phân riêng lẻ (TWF, HDF, PWF, OSF, RNF).

3 Vùng Lưu trữ Tạm (Staging Area) và Đường ống ETL

3.1 Lưu trữ Cơ sở dữ liệu

Một cơ sở dữ liệu PostgreSQL (co4031_dw) đã được thiết lập. Một schema staging và một bảng phẳng machine_data đã được tạo để chứa dữ liệu đã tiền xử lý trước khi tiến hành mô hình hóa chiều.

```

1 -- Create schema for staging area
2 CREATE SCHEMA IF NOT EXISTS staging;
3 DROP TABLE IF EXISTS staging.machine_data;
4

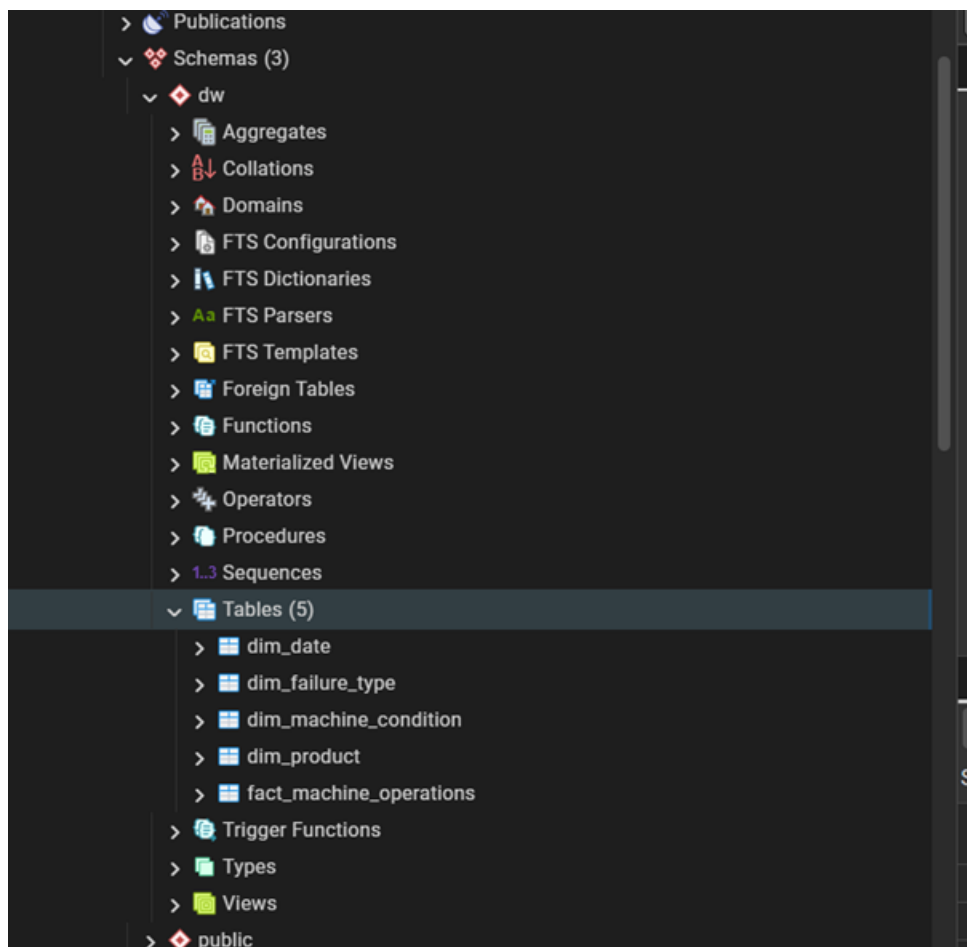
```

```

5  -- Create the staging table
6  CREATE TABLE staging.machine_data (
7      record_id INTEGER PRIMARY KEY,
8      product_id VARCHAR(20),
9      quality_type CHAR(1),
10     air_temp_k FLOAT,
11     process_temp_k FLOAT,
12     rotational_speed_rpm INTEGER,
13     torque_nm FLOAT,
14     tool_wear_min INTEGER,
15     machine_failure SMALLINT,
16     failure_twf SMALLINT,
17     failure_hdf SMALLINT,
18     failure_pwf SMALLINT,
19     failure_osf SMALLINT,
20     failure_rnf SMALLINT,
21     -- Derived / transformed columns
22     air_temp_c FLOAT,
23     process_temp_c FLOAT,
24     temp_differential_c FLOAT,
25     power_w FLOAT,
26     type_encoded SMALLINT,
27     failure_type VARCHAR(50),
28     load_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
29 );

```

Listing 2: Staging Schema



Hình 4: Tạo bảng staging trong PostgreSQL (pgAdmin)

3.2 Triển khai ETL

Quá trình ETL được thiết kế sử dụng hai phương pháp để đảm bảo tính linh hoạt:

- **Tải bằng Python:** Một tập lệnh (02_load_to_dw.py) sử dụng SQLAlchemy và pandas.to_sql được phát triển để chèn hàng loạt dữ liệu CSV đã làm sạch trực tiếp vào bảng staging.machine_data.

```
1 import pandas as pd
2 from sqlalchemy import create_engine, text
3 import warnings
4 warnings.filterwarnings('ignore')
5
6 # =====
7 # CONNECTION CONFIGURATION
8 # =====
9 # Change these if your PostgreSQL settings are different:
10 DB_USER = "postgres"
11 DB_PASSWORD = "885028" # your PostgreSQL password
12 DB_HOST = "localhost"
13 DB_PORT = "5432"
14 DB_NAME = "co4031_dw"
15
16 CLEANED_PATH = r"C:\co4031_project\data\cleaned\machine_data_cleaned.csv"
17
18 # Build connection string
19 conn_str = f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
20
21 print("Connecting to PostgreSQL...")
22 engine = create_engine(conn_str)
23
24 # Test connection
25 with engine.connect() as conn:
26     result = conn.execute(text("SELECT version()"))
27     print(f"Connected! PostgreSQL version: {result.fetchone()[0][:30]}...")
28
29 # =====
30 # LOAD DATA
31 # =====
32
33 print("\nLoading cleaned CSV file...")
34 df = pd.read_csv(CLEANED_PATH)
35 print(f"Loaded {len(df)} rows, {len(df.columns)} columns.")
36
37 # Select and rename columns to match database table exactly
38 db_columns = [
39     'record_id', 'product_id', 'quality_type',
40     'air_temp_k', 'process_temp_k', 'rotational_speed_rpm',
41     'torque_nm', 'tool_wear_min', 'machine_failure',
42     'failure_twf', 'failure_hdf', 'failure_pwf',
43     'failure_osf', 'failure_rnf',
44     'Air_Temp_C', 'Process_Temp_C',
45     'Temp_Differential_C', 'Power_W',
46     'Type_Encoded', 'Failure_Type'
47 ]
48
49 # Rename derived columns to lowercase for DB consistency
50 df = df.rename(columns={
51     'Air_Temp_C': 'air_temp_c',
52     'Process_Temp_C': 'process_temp_c',
53     'Temp_Differential_C': 'temp_differential_c',
54     'Power_W': 'power_w',
55     'Type_Encoded': 'type_encoded',
```

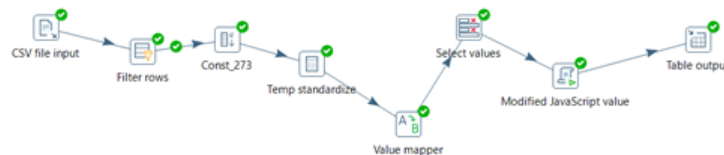
```

56     'Failure_Type': 'failure_type'
57 })
58
59 print("Loading data to staging.machine_data table...")
60
61 # Load using pandas to_sql (much faster than row-by-row INSERT)
62 df.to_sql(
63     name='machine_data',
64     schema='staging',
65     con=engine,
66     if_exists='replace', # 'replace' drops and recreates; use 'append' later
67     index=False,
68     chunksize=1000,      # process 1000 rows at a time
69     method='multi'       # batch inserts (faster)
70 )
71
72 # Verify the load
73 with engine.connect() as conn:
74     count = conn.execute(
75         text("SELECT COUNT(*) FROM staging.machine_data")
76     ).fetchone()[0]
77     print(f"\nVerification: {count} rows loaded to staging.machine_data")
78
79     # Show sample
80     sample = conn.execute(
81         text("SELECT record_id, quality_type, machine_failure, failure_type "
82             "FROM staging.machine_data LIMIT 5")
83     )
84     print("\nSample rows:")
85     for row in sample:
86         print(f"ID={row[0]}, Type={row[1]}, Failure={row[2]}, Kind='{row[3]}'")
87
88 print("\n=== ETL LOAD COMPLETE ===")

```

- **Pentaho Data Integration (Spoon):** Một đường ống ETL trực quan (machine_etl.ktr) đã được xây dựng. Các bước của đường ống bao gồm:

1. *CSV file input*: Đọc dữ liệu thô.
2. *Filter rows*: Lọc các bản ghi không hợp lệ (ví dụ: Tốc độ quay > 0).
3. *Calculator*: Chuẩn hóa nhiệt độ sang độ C và tính toán Chênh lệch Nhiệt độ và Công suất.
4. *Value Mapper*: Mã hóa loại chất lượng.
5. *Modified JavaScript Value*: Một tập lệnh để gộp các cờ lỗi thành một **failure_type** dễ đọc.
6. *Table output*: Đẩy luồng dữ liệu đã chuyển đổi vào bảng staging của PostgreSQL.



Hình 5: Quy trình ETL trực quan trên Pentaho Data Integration

Step name: **CSV file input**

Filename: **C:\co4031_project\data\awai4i2020.csv**

Delimiter: **,**

Enclosure: **"**

NIO buffer size: **50000**

Lazy conversion? ☐

Header row present? ☒

Add filename to result? ☐

The row number field name (optional):

Running in parallel? ☐

New line possible in fields? ☐

Format: **mixed**

File encoding:

#	Name	Type	Format	Length	Precision	Currency	Decimal	Group	Trim type
1	UDI	Integer	#	15	0	\$	-	-	none
2	Product ID	String		6		\$	-	-	none
3	Type	String		1		\$	-	-	none
4	Air temperature [K]	Number	##	5	1	\$	-	-	none
5	Process temperature [K]	Number	##	5	1	\$	-	-	none
6	Rotational speed [rpm]	Integer	#	15	0	\$	-	-	none
7	Torque [Nm]	Number	##	4	1	\$	-	-	none
8	Tool wear [min]	Integer	#	15	0	\$	-	-	none
9	Machine failure	Integer	#	15	0	\$	-	-	none
1.	TWF	Integer	#	15	0	\$	-	-	none
1.	HDF	Integer	#	15	0	\$	-	-	none
1.	PWF	Integer	#	15	0	\$	-	-	none
1.	CSF	Integer	#	15	0	\$	-	-	none
1.	RNF	Integer	#	15	0	\$	-	-	none
1.	UDI	Integer	#	15	0	\$	-	-	none

Buttons: **Help**, **OK**, **Get Fields**, **Review**, **Cancel**

Hình 6: Bước 1

Step name: **Filter rows**

Send 'true' data to step: **Const_273**

Send 'false' data to step:

The condition:

Rotational speed [rpm] **>** **0** (Integer)

Buttons: **Help**, **OK**, **Cancel**

Hình 7: Bước 2

Step name: **Const_273**

Fields:

#	Name	Type	Format	Length	Precision	Currency	Decimal	Group	Value	Set empty string?
1	Const_273	Number	0.00						273.15	N
2	pi_factor	Number	##0.###						0.10472	N

Buttons: **Help**, **OK**, **Cancel**

Hình 8: Bước 3

Calculator

Step name:

☒ Throw an error on non existing files

Fields:

#	New field	Calculation	Field A	Field B	Field C	Value type	Length	Precision	Remove	Conversion mask
1	Air_Temp_C	A - B	Air temperature [K]	Const_273		None			N	
2	Process_Temp_C	A - B	Process temperature [K]	Const_273		None			N	
3	Temp_Diff	A - B	Process temperature [K]	Air temperature [K]		None			N	
4	rpm_converted	A * B	Rotational speed [rpm]	pi_factor		None			N	
5	Power_W	A * B	Torque [Nm]	Rotational speed [rpm]		None			N	

Hình 9: Bước 4

Value mapper

Step name:

Fieldname to use:

Target field name (empty=overwrite):

Default upon non-matching:

Field values:

#	Source value	Target value
1	L	0
2	M	1
3	H	2

Hình 10: Bước 5

Select values

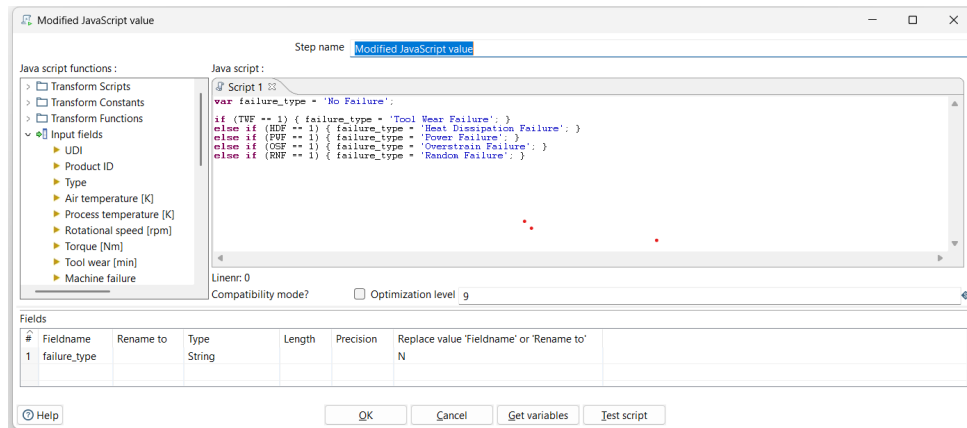
Step name:

Select & Alter / Remove / Meta-data

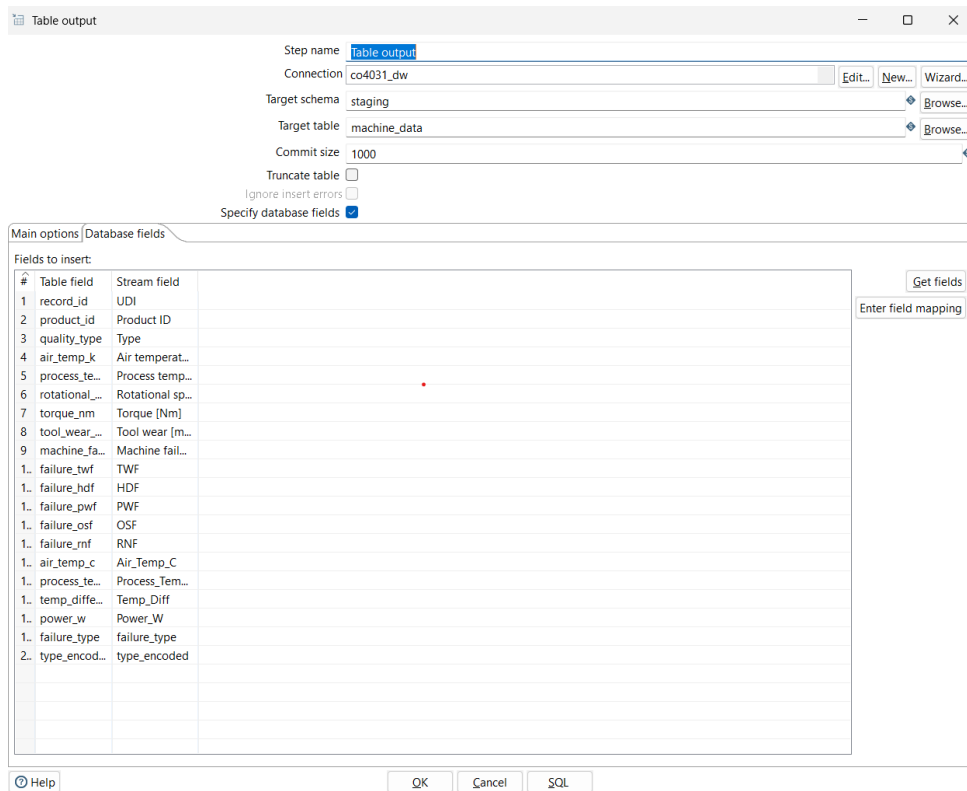
Fields to alter the meta-data for:

#	Fieldname	Rename to	Type	Length	Precision	Binary to Normal?	Format	Date Format Lenient?	Date L
1	type_encoded		Integer			N		N	

Hình 11: Bước 6



Hình 12: Bước 7



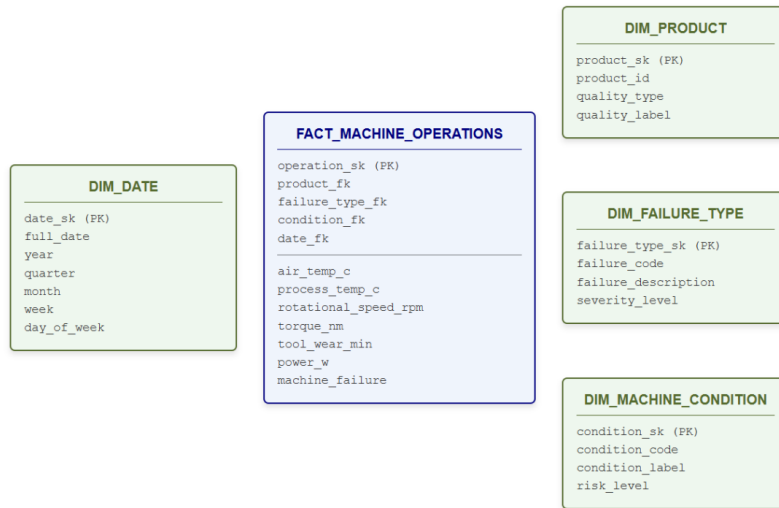
Hình 13: Bước 8

record_id	product_id	quality_type	air_temp_k	process_temp_k	rotational_speed_rpm	torque_nm	tool_wear_min	machine_failure
1	1	M14860	L	298.1	308.6	1551	42.8	0
2	2	L47181	L	298.2	308.7	1408	46.3	3
3	3	L47182	L	298.1	308.5	1498	49.4	5
4	4	L47183	L	298.2	308.6	1432	39.5	7
5	5	L49184	L	298.3	308.7	1408	48	9
6	6	M14865	M	298.1	308.6	1425	41.9	11
7	7	L47186	L	298.1	308.6	1558	42.4	14
8	8	L47187	L	298.1	308.6	1527	40.2	16

Hình 14: Sau khi populate dữ liệu vào staging

4 Thiết kế Kho Dữ Liệu: Lược đồ Hình sao (Star Schema)

Để hỗ trợ hiệu quả các truy vấn OLAP, dữ liệu phẳng trong vùng staging được mô hình hóa thành một Lược đồ Hình sao bao gồm một bảng sự kiện (fact table) và bốn bảng chiều (dimension tables).



Hình 15: Lược đồ Hình sao cho Kho Dữ liệu Sản xuất

4.1 Các Bảng Chiều (Dimension Tables)

- **DIM_DATE**: Chứa cấu trúc phân cấp thời gian (Năm, Quý, Tháng, Tuần). Vì dữ liệu thô thiếu dấu thời gian, một mô phỏng đã phân bổ các hoạt động qua các ngày làm việc trong năm 2023.
- **DIM_PRODUCT**: Lưu trữ các cấp chất lượng sản phẩm (Thấp, Trung bình, Cao).
- **DIM_FAILURE_TYPE**: Phân loại lỗi (ví dụ: Lỗi mòn dao cụ, Lỗi tản nhiệt) cùng với mức độ nghiêm trọng (từ 1 đến 3).
- **DIM_MACHINE_CONDITION**: Phân loại sức khỏe máy dựa trên số phút mòn của dao cụ (Dao mới, Dao ở giữa vòng đời, Dao cũ, Dao bị mòn) và gán mức độ rủi ro.

4.2 Bảng Sự kiện (Fact Table)

FACT_MACHINE_OPERATIONS nằm ở trung tâm của lược đồ. Nó chứa các khóa ngoại (foreign keys) liên kết tới các chiều đã nêu ở trên và lưu trữ các số đo liên tục như `air_temp_c`, `process_temp_c`, `torque_nm`, `tool_wear_min`, và `power_w`. Bảng này cũng lưu trữ các cờ lỗi nhị phân.

```
1 import pandas as pd
2 from sqlalchemy import create_engine, text
3 import numpy as np
4 from datetime import date, timedelta
5 import random
6 import warnings
7
8 warnings.filterwarnings('ignore')
9
10 # =====
11 # CONNECTION CONFIGURATION
12 # =====
13 DB_USER = "postgres"
14 DB_PASSWORD = "885028" # Update this to your actual password!
15 DB_HOST = "localhost"
16 DB_PORT = "5432"
17 DB_NAME = "co4031_dw"
18
19 conn_str = f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
20 engine = create_engine(conn_str)
21
22 print("Loading staging data...")
23 df = pd.read_sql("SELECT * FROM staging.machine_data", engine)
24 print(f"Loaded {len(df)} rows from staging.")
25
26 # =====
27 # POPULATE DIM_PRODUCT
28 # =====
29 print("\nPopulating dim_product ...")
30
31 quality_label_map = {
32     'L': 'Low Quality',
33     'M': 'Medium Quality',
34     'H': 'High Quality'
35 }
36 quality_tier_map = {'L': 1, 'M': 2, 'H': 3}
37
38 products = df[['product_id', 'quality_type']].drop_duplicates()
39 products['quality_label'] = products['quality_type'].map(quality_label_map)
40 products['quality_tier'] = products['quality_type'].map(quality_tier_map)
41
42 with engine.connect() as conn:
43     conn.execute(text("DELETE FROM dw.dim_product"))
44     conn.commit()
45
46 products.to_sql('dim_product', schema='dw', con=engine, if_exists='append',
47                 index=False)
48 print(f"Inserted {len(products)} product records.")
49
50 # Fetch back product SK mapping
51 dim_prod_df = pd.read_sql("SELECT product_sk, product_id FROM dw.dim_product",
52                             engine)
53 prod_map = dict(zip(dim_prod_df['product_id'], dim_prod_df['product_sk']))
54
55 # =====
```

```

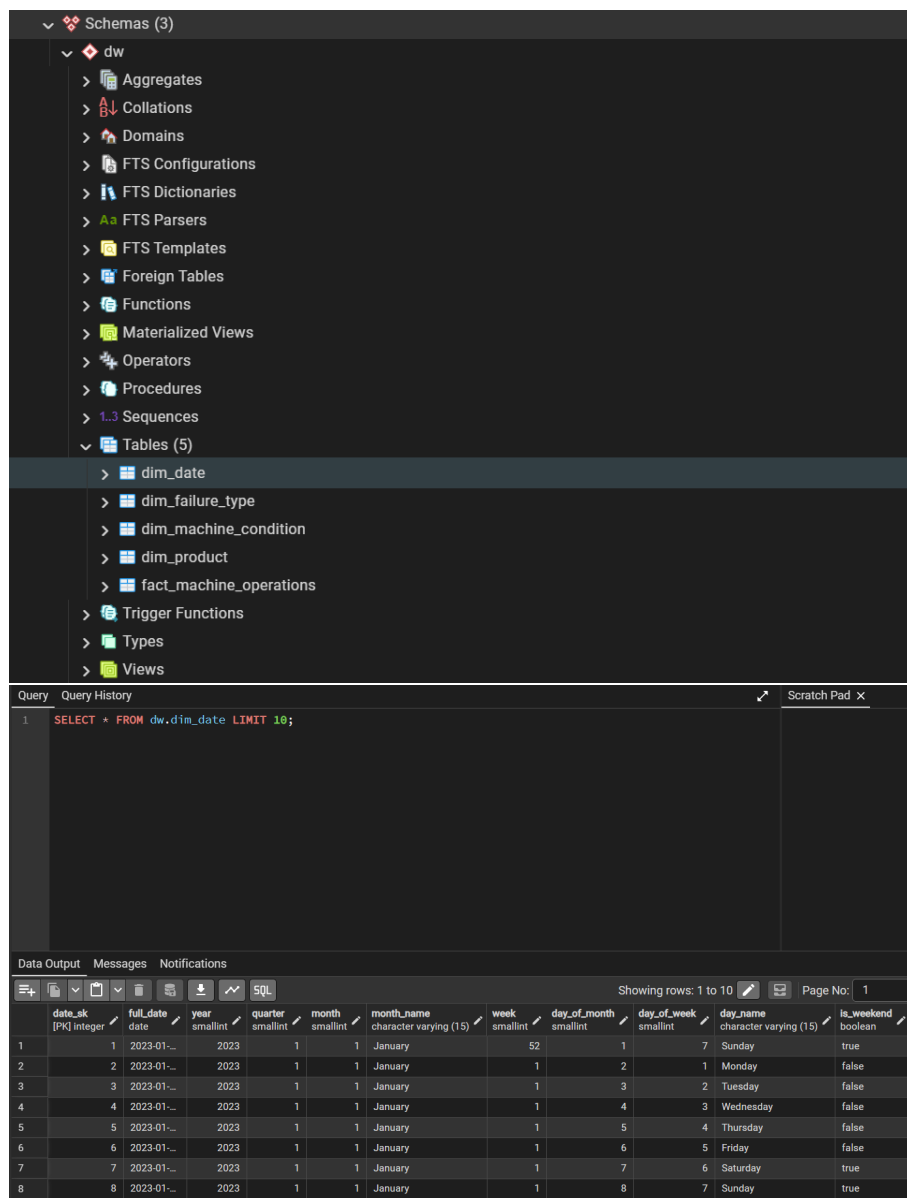
54 # POPULATE DIM_FAILURE_TYPE (already populated via SQL)
55 # =====
56 dim_fail_df = pd.read_sql("SELECT failure_type_sk, failure_code FROM dw.
    dim_failure_type", engine)
57 fail_code_map = dict(zip(dim_fail_df['failure_code'], dim_fail_df['
    failure_type_sk']))
58
59 def get_failure_code(row):
60     if row['failure_twf'] == 1: return 'TWF'
61     if row['failure_hdf'] == 1: return 'HDF'
62     if row['failure_pwf'] == 1: return 'PWF'
63     if row['failure_osf'] == 1: return 'OSF'
64     if row['failure_rnf'] == 1: return 'RNF'
65     return 'NF'
66
67 df['failure_code'] = df.apply(get_failure_code, axis=1)
68 df['failure_type_fk'] = df['failure_code'].map(fail_code_map)
69
70 # =====
71 # POPULATE DIM_MACHINE_CONDITION
72 # =====
73 dim_cond_df = pd.read_sql("SELECT condition_sk, condition_code FROM dw.
    dim_machine_condition", engine)
74 cond_map = dict(zip(dim_cond_df['condition_code'], dim_cond_df['condition_sk']))
75
76 def get_condition_code(wear_min):
77     if wear_min <= 100: return 'NEW_TOOL'
78     if wear_min <= 200: return 'MID_TOOL'
79     if wear_min <= 240: return 'OLD_TOOL'
80     return 'WORN_TOOL'
81
82 df['condition_code'] = df['tool_wear_min'].apply(get_condition_code)
83 df['condition_fk'] = df['condition_code'].map(cond_map)
84
85 # =====
86 # SIMULATE DATES (since dataset has no timestamps)
87 # =====
88 # We assign records to business days in 2023, simulating 10,000 operations
89 random.seed(42)
90 start_date = date(2023, 1, 2)
91 all_biz_days = []
92 d = start_date
93 while len(all_biz_days) < 400: # Ensure enough days
94     if d.weekday() < 5: # Monday-Friday
95         all_biz_days.append(d)
96         d += timedelta(days=1)
97
98 assigned_dates = [random.choice(all_biz_days) for _ in range(len(df))]
99 df['simulated_date'] = assigned_dates
100
101 # Fetch date dimension SK map
102 dim_date_df = pd.read_sql("SELECT date_sk, full_date FROM dw.dim_date", engine)
103 dim_date_df['full_date'] = pd.to_datetime(dim_date_df['full_date']).dt.date
104 date_map = dict(zip(dim_date_df['full_date'], dim_date_df['date_sk']))
105 df['date_fk'] = df['simulated_date'].map(date_map)
106
107 # =====
108 # POPULATE FACT TABLE
109 # =====
110 print("\nPopulating fact_machine_operations ...")
111
112 df['product_fk'] = df['product_id'].map(prod_map)
113

```

```

114 fact_cols = [
115     'product_fk', 'failure_type_fk', 'condition_fk', 'date_fk',
116     'record_id', # this will become source_record_id
117     'air_temp_c', 'process_temp_c', 'temp_differential_c',
118     'rotational_speed_rpm', 'torque_nm', 'tool_wear_min', 'power_w',
119     'machine_failure', 'failure_twf', 'failure_hdf', 'failure_pwf',
120     'failure_osf', 'failure_rnf'
121 ]
122
123 # Create fact dataframe and rename the source tracking ID
124 fact_df = df[fact_cols].copy()
125 fact_df = fact_df.rename(columns={'record_id': 'source_record_id'})
126
127 # Drop nulls in FKs (safety check)
128 before = len(fact_df)
129 fact_df = fact_df.dropna(subset=['product_fk', 'failure_type_fk', 'condition_fk',
130     , 'date_fk'])
131 dropped = before - len(fact_df)
132 if dropped > 0:
133     print(f"Warning: dropped {dropped} rows with null FK values")
134
135 # Load to fact table
136 with engine.connect() as conn:
137     conn.execute(text("DELETE FROM dw.fact_machine_operations"))
138     conn.commit()
139
140 fact_df.to_sql('fact_machine_operations', schema='dw', con=engine, if_exists='
141     append', index=False, chunksize=500, method='multi')
142
143 # Verify
144 with engine.connect() as conn:
145     cnt = conn.execute(text("SELECT COUNT(*) FROM dw.fact_machine_operations")).
146     fetchone()[0]
147     print(f"Inserted {cnt} rows into fact_machine_operations.")
148
149     fail_cnt = conn.execute(text("SELECT COUNT(*) FROM dw.
150     fact_machine_operations WHERE machine_failure = 1")).fetchone()[0]
151     print(f"Of which {fail_cnt} are failure records.")
152
153 print("\n=== STAR SCHEMA POPULATION COMPLETE ===")

```



Hình 16: Dữ liệu đã được nạp thành công vào Schema

5 Phân tích OLAP

Với Lược đồ Hình sao đã được nạp dữ liệu, một số truy vấn OLAP đã được phát triển để trích xuất thông tin tình báo kinh doanh.

5.1 Truy vấn 1: Tỷ lệ Lỗi theo Loại Chất lượng Sản phẩm

Phân tích độ tin cậy của các cấp chất lượng sản phẩm khác nhau (L, M, H) bằng cách tính tổng số hoạt động, tỷ lệ lỗi, mô-men xoắn trung bình và độ mòn trung bình của dao cụ cho mỗi cấp.

Query

Query History

```

1  -- OLAP Query 1: Failure Rate by Product Quality Type
2  SELECT
3      p.quality_label AS "Product Quality",
4      COUNT(*) AS "Total Operations",
5      SUM(f.machine_failure) AS "Total Failures",
6      ROUND((100.0 * SUM(f.machine_failure) / COUNT(*)::numeric, 2) AS "Failure Rate (%)",
7      ROUND(AVG(f.torque_nm)::numeric, 2) AS "Avg Torque (Nm)",
8      ROUND(AVG(f.tool_wear_min)::numeric, 1) AS "Avg Tool Wear (min)"
9  FROM dw.fact_machine_operations f
10 JOIN dw.dim_product p ON f.product_fk = p.product_sk
11 GROUP BY p.quality_label, p.quality_tier
12 ORDER BY p.quality_tier;

```

Data Output

Messages

Notifications

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

+

Hình 17: Tỷ lệ Lỗi theo Loại Chất lượng Sản phẩm

5.2 Truy vấn 2: Xu hướng Lỗi Hàng tháng

Sử dụng bảng DIM_DATE để nhóm các hoạt động theo năm và tháng. Truy vấn này tính toán tỷ lệ lỗi hàng tháng, cung cấp cái nhìn rõ ràng về các sai lệch hoạt động có thể xảy ra theo mùa hoặc theo thời gian.

Query

Query History

1

-- OLAP Query 2: Monthly Failure Trend

2

SELECT

3

d.year AS "Year",

4

d.month AS "Month",

5

d.month_name AS "Month Name",

6

COUNT(*) AS "Operations",

7

SUM(f.machine_failure) AS "Failures",

8

ROUND(100.0 * SUM(f.machine_failure) / COUNT(*), 2) AS "Failure Rate (%)",

9

FROM dw.fact_machine_operations f

10

JOIN dw.dim_date d ON f.date_fk = d.date_sk

11

GROUP BY d.year, d.month, d.month_name

12

ORDER BY d.year, d.month;

Data Output

Messages

Notifications

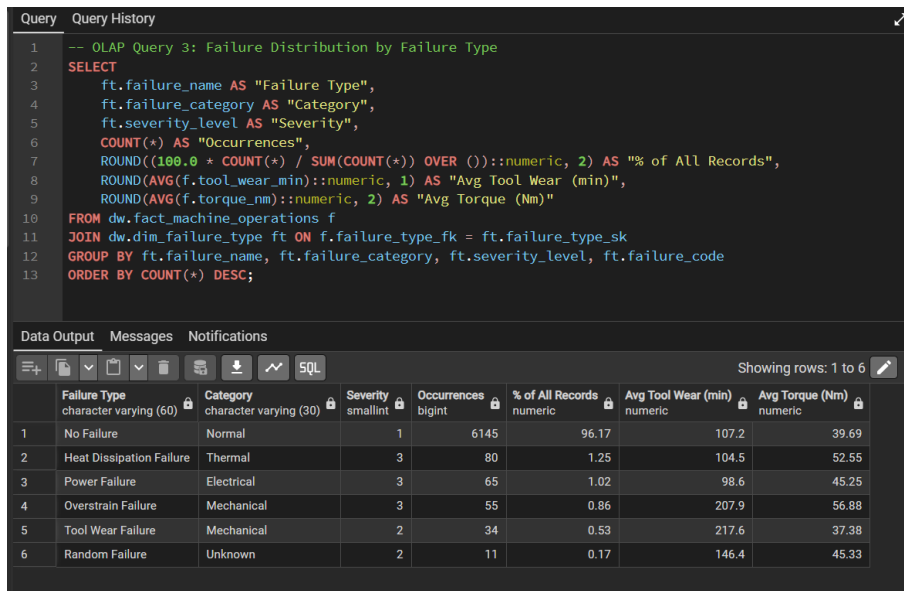
SQL

	Year smallint	Month smallint	Month Name character varying (15)	Operations bigint	Failures bigint	Failure Rate (%) numeric
1	2023	1	January	490	18	3.67
2	2023	2	February	502	17	3.39
3	2023	3	March	585	23	3.93
4	2023	4	April	516	17	3.29
5	2023	5	May	533	23	4.32
6	2023	6	June	526	19	3.61
7	2023	7	July	501	23	4.59
8	2023	8	August	590	20	3.39

Hình 18: Kết quả truy vấn xu hướng lỗi hàng tháng

5.3 Truy vấn 3: Phân phối Lỗi theo Loại Lỗi

Tổng hợp dữ liệu theo các danh mục lỗi được định nghĩa trong DIM_FAILURE_TYPE. Truy vấn này tính toán số lần xuất hiện và tỷ lệ phần trăm trên tổng số lỗi, làm nổi bật xem lỗi cơ học, lỗi nhiệt, hay lỗi điện là phổ biến nhất.



The screenshot shows a SQL query editor with a query titled "OLAP Query 3: Failure Distribution by Failure Type". The query selects failure details and calculates occurrence counts and percentages. Below the query, a table displays the results for six failure types.

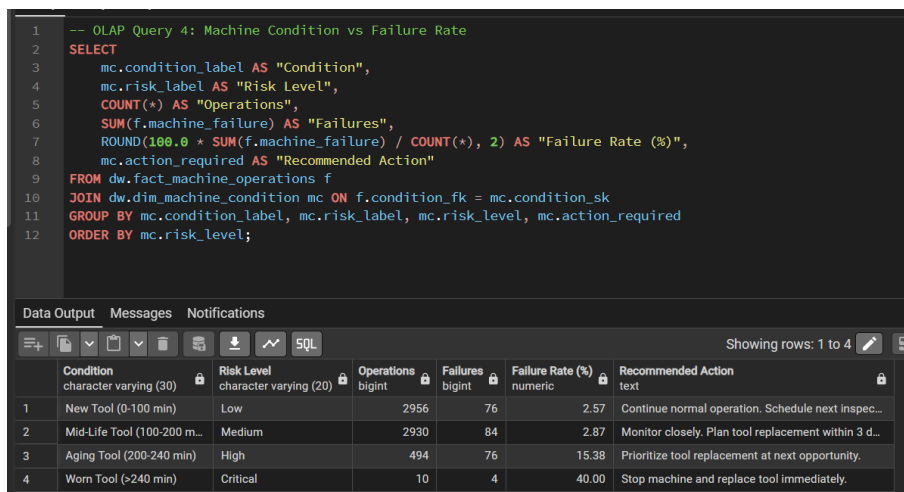
```
1 -- OLAP Query 3: Failure Distribution by Failure Type
2 SELECT
3     ft.failure_name AS "Failure Type",
4     ft.failure_category AS "Category",
5     ft.severity_level AS "Severity",
6     COUNT(*) AS "Occurrences",
7     ROUND((100.0 * COUNT(*) / SUM(COUNT(*) OVER ()):numeric, 2) AS "% of All Records",
8     ROUND(AVG(f.tool_wear_min)::numeric, 1) AS "Avg Tool Wear (min)",
9     ROUND(AVG(f.torque_nm)::numeric, 2) AS "Avg Torque (Nm)"
10 FROM dw.fact_machine_operations f
11 JOIN dw.dim_failure_type ft ON f.failure_type_fk = ft.failure_type_sk
12 GROUP BY ft.failure_name, ft.failure_category, ft.severity_level, ft.failure_code
13 ORDER BY COUNT(*) DESC;
```

Failure Type	Category	Severity	Occurrences	% of All Records	Avg Tool Wear (min)	Avg Torque (Nm)
No Failure	Normal	1	6145	96.17	107.2	39.69
Heat Dissipation Failure	Thermal	3	80	1.25	104.5	52.55
Power Failure	Electrical	3	65	1.02	98.6	45.25
Overstrain Failure	Mechanical	3	55	0.86	207.9	56.88
Tool Wear Failure	Mechanical	2	34	0.53	217.6	37.38
Random Failure	Unknown	2	11	0.17	146.4	45.33

Hình 19: Kết quả truy vấn xu hướng lỗi hàng tháng

5.4 Truy vấn 4: Tình trạng Máy vs Tỷ lệ Lỗi

Sử dụng bảng DIM_MACHINE_CONDITION để đánh giá xem tuổi của dao cụ ảnh hưởng như thế nào đến xác suất xảy ra lỗi. Truy vấn vạch ra các hoạt động trên nhiều mức độ rủi ro khác nhau (Thấp, Trung bình, Cao, Nghiêm trọng) và liên kết chúng với các đề xuất bảo trì có thể hành động.



The screenshot shows a SQL query editor with a query titled "OLAP Query 4: Machine Condition vs Failure Rate". The query selects machine condition details and calculates failure rates. Below the query, a table displays the results for four machine conditions.

```
1 -- OLAP Query 4: Machine Condition vs Failure Rate
2 SELECT
3     mc.condition_label AS "Condition",
4     mc.risk_label AS "Risk Level",
5     COUNT(*) AS "Operations",
6     SUM(f.machine_failure) AS "Failures",
7     ROUND(100.0 * SUM(f.machine_failure) / COUNT(*), 2) AS "Failure Rate (%)",
8     mc.action_required AS "Recommended Action"
9 FROM dw.fact_machine_operations f
10 JOIN dw.dim_machine_condition mc ON f.condition_fk = mc.condition_sk
11 GROUP BY mc.condition_label, mc.risk_label, mc.risk_level, mc.action_required
12 ORDER BY mc.risk_level;
```

Condition	Risk Level	Operations	Failures	Failure Rate (%)	Recommended Action
New Tool (0-100 min)	Low	2956	76	2.57	Continue normal operation. Schedule next inspec...
Mid-Life Tool (100-200 m...	Medium	2930	84	2.87	Monitor closely. Plan tool replacement within 3 d...
Aging Tool (200-240 min)	High	494	76	15.38	Prioritize tool replacement at next opportunity.
Worn Tool (>240 min)	Critical	10	4	40.00	Stop machine and replace tool immediately.

Hình 20: Kết quả truy vấn xu hướng lỗi hàng tháng

6 Kết luận

Dự án đã chuyển đổi thành công dữ liệu cảm biến máy thô thành một Kho dữ liệu có cấu trúc, tối ưu cho truy vấn. Việc triển khai các bước tiền xử lý mạnh mẽ, một đường ống ETL có hệ

thống (qua Python và Pentaho), và mô hình hóa chiều cho phép theo dõi sức khỏe máy hiệu quả và phân tích bảo trì dự đoán thông qua các truy vấn OLAP toàn diện.